

JAVA INTERVIEW PREPARATION

CHAPTER 27

JVM RUNTIME MEMORY

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

JVM Runtime Memory: Heap, Stacks, Metaspace, Code Cache, and Native Memory

Senior / Lead Java Developer Reading Chapter

When a Java process consumes memory, the Java heap is only one part of the total. Every thread needs stack and runtime bookkeeping, loaded classes require metadata, JIT-compiled methods occupy a code cache, direct buffers use native memory, and the JVM itself maintains GC and internal structures. Senior-level diagnosis begins by asking which memory area is growing rather than assuming that every large resident process has a heap leak.

Specification Areas and HotSpot Implementation

The Java Virtual Machine Specification defines logical runtime data areas such as the heap, per-thread JVM stacks, the method area, the runtime constant pool, native method stacks, and the program counter. It deliberately allows implementation freedom.

HotSpot realizes these concepts using concrete mechanisms such as the Java heap, Metaspace, compressed class space, code cache, native thread stacks, and internal native allocations. Metaspace is therefore a HotSpot implementation of class-metadata storage, not the definition of the JVM specification's method area.

```
Java process address space
+-- Java heap: objects and arrays
+-- Metaspace / compressed class space: class metadata
+-- Code cache: interpreted and JIT-compiled code support
+-- Thread stacks: frames, local variables, operand stacks
+-- Direct/native buffers and JNI allocations
+-- GC, compiler, symbol, arena, and JVM internal memory
+-- Mapped files, shared libraries, and operating-system overhead
```

The Java Heap

The heap is shared by all threads and is the primary storage area for ordinary objects and arrays. Garbage collection reclaims heap objects that are no longer reachable through the GC root graph.

```
Order order = new Order(id, items);
```

The local variable `order` is conceptually part of a stack frame, while the `Order` object and its referenced arrays or collections normally live on the heap. The JIT compiler may eliminate an allocation, scalar-replace an object, or keep values in registers, so this picture is a reasoning model rather than a promise about every optimized machine instruction.

`-Xms` sets an initial heap-sizing target and `-Xmx` sets the maximum heap. Committed heap is memory currently made available to the JVM; used heap contains current allocations; reserved address space may be larger than committed memory.

```
reserved heap: [-----]
committed:   [-----]
used:        [-----]
```

Operating-system resident memory does not need to equal used heap. Pages, native areas, allocator behavior, shared mappings, and container accounting all influence RSS.

Object Reachability and GC Roots

Garbage collection follows references from roots. Typical roots include active thread stacks, static fields of loaded classes, JNI references, and JVM internal structures.

```
GC root -> Service -> Cache -> Map entries -> retained objects

unreachable cycle: A -> B -> C -> A
                  no path from a root, therefore collectible
```

Java GC is based on reachability, not reference counting. A cycle is collectible when no root can reach it. A memory leak occurs when an unnecessary path from a root remains, not merely because objects reference one another.

Thread Stacks and Frames

Each platform thread has its own JVM stack. A method invocation creates a frame containing local variables, an operand stack, return information, and implementation bookkeeping.

```
Thread stack
+-- handleRequest frame
|   locals: request, customerId
+-- loadCustomer frame
|   locals: repository, id
+-- decode frame
    locals and operand stack
```

Primitive local values can reside directly in a frame. Object locals hold references, not complete heap objects. A reference in a live frame can keep a large object graph reachable until the reference is cleared, overwritten, or the frame becomes inactive.

Deep or infinite recursion can exhaust stack capacity and produce `StackOverflowError`. `-Xss` influences platform-thread stack size. Increasing it allows deeper stacks but raises native memory reserved per thread and can reduce the number of platform threads the process can create.

Virtual threads do not reserve one traditional large OS-thread stack each. Their stacks are represented in heap-managed stack chunks when unmounted and grow as required. This improves the cost of large waiting-thread populations, but captured task state and `ThreadLocal` values still consume memory.

StackOverflowError Is Not OutOfMemoryError

```
void recurse() {
    recurse();
}
```

`StackOverflowError` normally means one thread exceeded its stack, often through uncontrolled recursion.

`OutOfMemoryError: unable to create native thread` points instead to native thread or operating-system resource exhaustion. A process can have substantial free Java heap and still fail to create a platform thread.

Fix the recursion or unbounded thread creation before increasing limits. Larger limits can postpone the same defect and increase failure impact.

Metaspace and Class Metadata

HotSpot stores most class metadata in native memory called Metaspace. Metadata is associated with class loaders. Classes can normally be unloaded only when their defining class loader becomes unreachable and other unloading conditions are satisfied.

```
application class loader
+-- Class metadata for ServiceA
+-- Class metadata for ServiceB
+-- static fields and related Class objects
```

`-XX:MaxMetaspaceSize` optionally caps Metaspace. Without a practical cap, native memory or container limits can be reached first. `OutOfMemoryError: Metaspace` may indicate too many legitimately loaded classes, excessive runtime generation, or a class-loader leak.

On suitable 64-bit HotSpot configurations, compressed class pointers use a separate compressed class space for part of class metadata. `OutOfMemoryError: Compressed class space` is distinct from ordinary Java heap exhaustion.

Runtime Constant Pools and String Pool Placement

Each loaded class or interface has a runtime constant pool derived from symbolic information in its class file. This is not the same thing as the String intern pool.

Interned Strings are ordinary String objects managed in the heap in modern HotSpot. The pool maintains canonical references for interned values. Class-file constants, runtime constant pools, String objects, and class metadata are related during execution but should not be described as one physical pool.

Code Cache and JIT Compilation

HotSpot begins by interpreting code and compiles frequently executed methods to native machine code. Compiled methods and related structures occupy the code cache.

```
bytecode -> interpreter -> profiling -> JIT compilation -> native code in code cache
```

Code-cache pressure can stop or limit compilation and reduce performance even when heap is healthy. Compilation logs, JFR, and code-cache diagnostics help distinguish this from application CPU problems. The exact tiered-compilation and cache layout are implementation details.

Direct Buffers and Off-Heap Memory

`ByteBuffer.allocateDirect` requests memory outside the Java heap, normally through native allocation managed by a small heap object and a cleanup mechanism.

```
ByteBuffer buffer = ByteBuffer.allocateDirect(16 * 1024 * 1024);
```

Direct buffers can reduce copying for some I/O operations, but their native memory is not visible as ordinary heap payload. Their reclamation depends on reachability and cleanup timing, so treating them as an unlimited manual heap is unsafe.

`-XX:MaxDirectMemorySize` influences direct-buffer limits in HotSpot. Libraries such as networking frameworks may use their own pooling and native-allocation strategies, so JVM direct-buffer counters may not explain every off-heap byte.

Memory-mapped files add another distinction: mapped address space and resident pages are managed with operating-system involvement and do not behave like a normal heap byte array.

Native Threads and Per-Thread Cost

Every platform thread consumes native stack space and operating-system resources. A service with thousands of platform threads can exhaust native memory even with a modest heap.

```
process memory budget
- Java heap
- metaspace and code cache
- native thread stacks x thread count
- direct buffers
- JVM and native libraries
```

Setting `-Xmx` equal to a container's entire memory limit leaves no headroom for these areas. Container termination by the operating system may occur without a Java heap `OutOfMemoryError` or heap dump.

Native Memory Tracking

HotSpot Native Memory Tracking, or NMT, categorizes JVM-managed native memory. It must be enabled at startup, commonly with `-XX:NativeMemoryTracking=summary` or `detail`, and has overhead.

```
jcmd <pid> VM.native_memory summary
jcmd <pid> VM.native_memory baseline
jcmd <pid> VM.native_memory summary.diff
```

NMT reports reserved and committed memory by JVM subsystem. It does not track every allocation made by arbitrary native application code, so unexplained growth may still require operating-system tools, native profilers, or library-specific metrics.

Escape Analysis and Allocation Elimination

The JIT may determine that an object does not escape a method or thread and eliminate allocation or replace fields with scalar values.

```
Point point = new Point(x, y);
return point.x() + point.y();
```

Do not claim that every local object is allocated on the stack. The language and JVM specification do not promise stack allocation for such code. Say that HotSpot may optimize away allocations based on escape analysis. Confirm actual behavior with profiling rather than source-level guesses.

Allocation Is Usually Cheaper Than Retention

In a generational collector, many small objects are allocated through a fast pointer bump in a thread-local allocation buffer, or TLAB. Allocation can therefore be extremely cheap. The expensive outcomes are often retaining objects too long, copying or scanning them repeatedly, promoting them into older regions, or allocating at a rate the collector cannot sustain.

```
thread -> TLAB -> fast sequential allocations
|
+--> objects die young: inexpensive reclamation
+--> objects remain reachable: copied/promoted/retained
```

This is why object pooling ordinary short-lived Java objects can make performance worse. A pool adds synchronization and keeps objects reachable, increasing the live set. Pool scarce external resources such as database connections when their creation and capacity justify it; do not automatically pool simple domain objects.

Large allocations behave differently from tiny ones, and details vary by collector. Arrays, buffers, or serialized payloads can create allocation spikes, region pressure, or long copying work. Diagnose allocation rate, object lifetime, and retained size rather than counting `new` expressions in source.

Shallow Size, Retained Size, and Dominators

An object's shallow size is the memory occupied by the object itself. Its retained size estimates what could become collectible if that object and the objects exclusively reachable through it were removed. A small map or class-loader object can therefore dominate gigabytes of entries or metadata.

```
GC root
-> SessionRegistry      small shallow size
    -> Map
        -> 2,000,000 sessions

SessionRegistry dominates the retained graph
```

A dominator is a node through which every path from a GC root to another object passes. Heap analyzers use dominator trees to identify ownership of retained memory. The largest object by shallow size is rarely the whole answer; inspect paths to GC roots and dominators to find why the graph remains reachable.

Heap histograms are useful for fast trend comparison but do not show ownership by themselves. Millions of `byte[]` instances might be HTTP bodies, cache values, class data, strings, or buffers. A heap dump adds reference paths but can be large, expensive to produce, and may contain sensitive application data. Capture it under an operationally approved procedure.

Common Root Paths That Become Leaks

Java leaks are usually valid references with invalid lifetimes:

```
static collection -> entries never removed
ThreadLocal on pooled thread -> request object graph
listener registry -> listener and owning component
cache -> unbounded keys or missing weight/expiry
ClassLoader -> classes -> static fields -> application graph
pending queue/Future -> captured task arguments
JNI global reference -> heap object
```

The fix follows ownership. Bound caches by a business capacity and use appropriate eviction. Remove listeners when the owner closes. Clear or restore thread-local values in `finally`. Cancel and remove abandoned tasks. Make lifecycle boundaries explicit instead of using weak references as a universal patch. A weak key does not help when the value strongly references that key or another root retains the graph.

Reserved, Committed, and Resident Are Different

Virtual address space is not the same as physical memory. The JVM can reserve a large address range, commit part of it for use, and have only some committed pages resident in RAM at a given instant.

```
reserved address range [-----]
committed pages        [-----]
currently resident      [-----]
```

Operating-system tools may report virtual size, resident set, private dirty pages, shared mappings, and cgroup usage differently. Compare like with like. An NMT "reserved" value should not be added directly to RSS as if every page were physically resident. Similarly, dropping heap usage after GC may not immediately reduce RSS because the JVM can keep committed pages for reuse and because collector and operating-system uncommit policies differ.

An Explicit Container Memory Budget

Production sizing should begin with the process limit and allocate headroom deliberately:

```
container limit
- maximum/committed Java heap
- metaspace and compressed class space
- code cache
- platform-thread stacks
- direct and pooled native buffers
- GC and compiler structures
- JNI/native libraries and allocator fragmentation
- diagnostic and safety headroom
```

If `-Xmx` consumes almost the entire limit, the operating system can kill the process when native usage grows. That kill may leave no Java `OutOfMemoryError`, heap dump, or orderly shutdown log. Container metrics and termination reason are therefore part of JVM memory evidence.

Percent-based heap sizing can adapt to a detected container limit, but it is still a policy that must leave space for workload-specific native areas. A service with 2,000 platform threads or a large direct-buffer pool needs more headroom than a single-threaded batch with the same heap live set.

OutOfMemoryError Messages Point to Different Exhaustion

`OutOfMemoryError` is a family of resource failures, not one diagnosis:

Java heap space	allocation cannot be satisfied in heap
GC overhead limit exceeded	excessive collection with little recovery
Metaspace	class metadata cannot grow
Compressed class space	compressed class-pointer region exhausted
Direct buffer memory	direct-buffer limit/allocation pressure
unable to create native thread	OS/native resources cannot create thread

Exact messages are implementation-dependent, but the category determines the first evidence to collect. A heap dump helps heap retention; it does not explain a missing OS thread quota. A class histogram is not enough for native growth. Avoid increasing a limit until the growth curve and ownership are understood.

Automatic heap dumps on heap OOM can be valuable, but disk capacity, pause impact, sensitive content, and collection location must be planned before the incident. Native or container exhaustion may prevent the dump or bypass that mechanism entirely.

Native Memory Tracking as a Difference Tool

NMT is most useful when treated as a trend rather than one absolute snapshot:

```
-XX:NativeMemoryTracking=summary

jcmd <pid> VM.native_memory baseline
# wait through representative load
jcmd <pid> VM.native_memory summary.diff scale=MB
```

The difference report can show that thread, class, code, GC, or another HotSpot category is growing. Summary mode has less detail; detail mode can associate more data with allocation sites but costs more. NMT must be enabled at startup. It tracks HotSpot/JVM-managed native allocations, not every third-party JNI or native-library allocation, so a large RSS remainder may require operating-system maps or a native profiler.

A Production Investigation Sequence

When memory alerts fire, first establish whether the symptom is heap usage, post-GC live set, allocation rate, RSS, container usage, native commitment, or process termination. Then correlate the timeline with deployment, traffic, thread count, loaded classes, cache population, direct buffers, and GC behavior.

For a suspected heap leak, compare histograms or dumps across time and inspect dominators and GC-root paths. For Metaspace growth, trend loaded and unloaded classes and class-loader instances. For native growth, compare NMT baselines, thread stacks, direct-buffer metrics, and OS mappings. For container kills, reconstruct the total budget rather than inspecting only `-Xmx`.

Mitigation depends on the area: reduce admission, disable a leaking feature, bound a cache, lower platform-thread concurrency, or restart when required to restore service. Preserve evidence before restart when safe. The permanent correction is an ownership or capacity change, not merely a larger memory ceiling.

A Strong Interview Explanation

Begin by separating specification concepts from HotSpot implementation. The heap, per-thread stacks, method area, and runtime constant pools are JVM concepts; Metaspace, compressed class space, code cache, TLABs, and NMT are HotSpot mechanisms. Then explain that process RSS includes heap plus native stacks, class metadata, compiled code, direct buffers, GC structures, libraries, mappings, and allocator behavior.

For leaks, describe reachability from GC roots, dominators, retained size, and invalid object lifetimes. For diagnosis, use post-GC live-set and allocation trends, heap dumps, class-loader metrics, NMT differences, JFR, and OS/container evidence. State that virtual threads reduce per-waiting-thread native stack cost but still retain heap state, and that a container must reserve headroom beyond the heap. This demonstrates a memory-budget model rather than a memorized list of regions.

Production Memory Accounting

When RSS is high, compare:

- Heap used, committed, maximum, and post-GC live set.
- Metaspace and loaded-class trends.
- Platform and virtual thread counts.
- Direct-buffer pools and native library metrics.
- Code cache and compiler activity.
- NMT categories and their change from baseline.
- Container limit, page cache, mapped files, and OS process accounting.

A flat heap graph with growing RSS points away from an ordinary heap leak. A growing post-GC live set points toward retained heap data. Growing Metaspace plus increasing class-loader counts suggests dynamic class generation or loader retention.

Interview-Ready Summary

The Java heap stores ordinary objects and is only one part of process memory. Platform threads have native stacks; HotSpot stores class metadata in Metaspace and compiled code in code cache; direct buffers and JVM subsystems consume native memory. Virtual-thread stacks use heap-managed chunks rather than one large native stack per virtual thread.

Garbage collection follows reachability from roots. Cycles are collectible when unreachable. High RSS does not prove a heap leak, and a free heap does not prevent native-thread, Metaspace, direct-memory, or container-limit failure. Diagnose the correct memory area with heap evidence, class-loader metrics, NMT, JFR, and OS data.

Revision Notes

- JVM specification areas and HotSpot implementation areas are not identical terms.
- Heap is shared and normally stores objects and arrays.
- Used, committed, reserved, and resident memory mean different things.
- GC follows reachability from roots rather than reference counts.
- Unreachable cycles can be collected.
- Each platform thread has a native-backed JVM stack.
- Stack frames contain locals, operand stacks, and call bookkeeping.
- Object locals contain references, not whole objects.
- Deep recursion can cause StackOverflowError.
- Larger -Xss increases per-platform-thread memory cost.
- Virtual-thread stacks use heap-managed chunks when unmounted.
- HotSpot stores class metadata in native Metaspace.
- Metaspace exhaustion can indicate class-loader retention.
- Runtime constant pools and the String intern pool are different concepts.
- JIT-compiled methods consume code-cache memory.
- Direct buffers consume memory outside the Java heap.
- A container needs headroom beyond -Xmx.
- NMT categorizes JVM-managed native memory when enabled at startup.
- NMT does not account for every arbitrary native allocation.
- Escape analysis may eliminate allocation; Java does not promise local objects live on stacks.
- Diagnose high memory by area and trend, not by heap size alone.
- Fast allocation does not make long retention inexpensive.
- TLABs make many thread-local small allocations cheap.
- Retained size and dominators reveal ownership better than shallow size alone.
- Heap histograms count classes but do not explain root ownership.
- Java leaks are usually valid references with incorrect lifetimes.
- Reserved address space, committed memory, and RSS are different measurements.
- Budget container memory for heap plus every important native category.
- OOM messages identify different resource areas and require different evidence.
- Use NMT baseline differences to identify growing HotSpot native categories.
- Heap dumps may be expensive and contain sensitive data.